

Random Forest Hyperparameters

Contents

1. Overview	2
2. The three groups	3
3. The forest-level hyperparameters	3
3.1 The four settings	3
3.2 How many features <code>max_features</code> means	4
3.3 Trying the settings on a demo dataset	4
4. The tree-level hyperparameters	6
5. The general hyperparameters	7
6. The random forest regressor	8
7. Summary	10
8. Sources	10
9. Key terms	10

Where this fits: Pipeline map steps 8 (Model), 10 (Tune). See the [Course map](#).



- **Builds on:** Overfitting ([Note 91](#)); Decision trees ([Note 100](#)); Feature importance ([Note 100](#)); Bagging ([Note 107](#)); OOB score ([Note 107](#)); Grid and random search ([Note 107](#)).
- **Leads to:** Grid and random search ([Note 112](#)); AdaBoost ([Note 115](#)); Balanced random forest ([Note 133](#)); Optuna ([Note 134](#)); Improving a neural network ([Note 1021](#)).
- **Compare with:** Bagging ([Note 107](#)); Dropout ([Note 1024](#)).

1. Overview

Key point: A random forest's hyperparameters fall into three groups: four that shape the forest, the usual decision tree settings for each tree, and a few general ones. Only the first group is new.

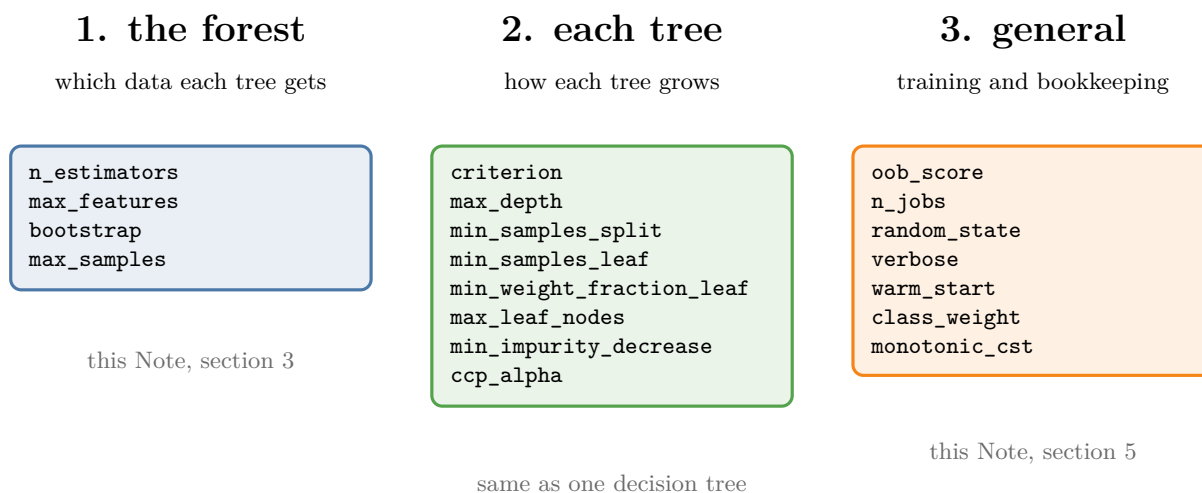


Figure 1: The hyperparameters of RandomForestClassifier, in three groups

A random forest (the [random forest introduction Note](#)) is a very flexible algorithm with many hyperparameters. Its trees learn from **observations** (records, one row of the data table each); each observation has **features** (input variables, one column each) and a **target** (the output we predict). Figure 1 sorts them into three groups. `RandomForestClassifier` and `RandomForestRegressor` have nearly the same settings, so learning the classifier's covers the regressor too (section 6).

The Notebook (`notebook.ipynb`) runs every experiment. The Dash app `app.py` has a control for each of the four forest-level settings and redraws the decision surface with its test accuracy.

2. The three groups

Key point: Forest settings decide which data each tree gets; tree settings decide how each tree grows; general settings handle training and bookkeeping.

1. **Forest-level** (section 3): `n_estimators`, `max_features`, `bootstrap`, `max_samples`. These tune the forest itself.
2. **Tree-level** (section 4): `criterion`, `max_depth`, `min_samples_split` and the rest. A forest is built from decision trees, and these are the settings of every one of them.
3. **General** (section 5): `n_jobs`, `random_state`, `verbose` and others found in many scikit-learn algorithms.

3. The forest-level hyperparameters

Key point: `n_estimators` sets the number of trees, `max_samples` the observations per tree, `max_features` the features considered at each split, and `bootstrap` whether observations are drawn with replacement.

3.1 The four settings

Key point: Two settings size the forest and its samples; two decide how observations and features are drawn.

Setting	Default	Meaning
<code>n_estimators</code>	100	the number of trees in the forest
<code>max_features</code>	"sqrt" (classifier), 1.0 (regressor)	how many features each split considers
<code>bootstrap</code>	True	draw each tree's observations with replacement (True) or give every tree the whole training set (False)
<code>max_samples</code>	None (as many observations as the training set)	how many observations each tree gets; only used when <code>bootstrap=True</code>

`max_features` and `max_samples` accept a whole number (a count) or a decimal (a share), as in `BaggingClassifier` (the [bagging classifier Note](#), section 3.2).

3.2 How many features `max_features` means

Key point: “sqrt” takes the square root of the number of features, “log2” its base-2 logarithm, a decimal a share, and `None` all of them; each rounds down.

At every split, a tree draws `max_features` features at random and picks the best split among them (node-level sampling: the [bagging vs random forest Note](#), section 3.2). With p features:

1. **In words:** turn the setting into a number of features, then round down (but never below 1).
2. **Formula:**

$$\text{"sqrt"} : \lfloor \sqrt{p} \rfloor \quad \text{"log2"} : \lfloor \log_2 p \rfloor \quad \text{decimal } f : \lfloor f \cdot p \rfloor \quad \text{None} : p$$

where $\lfloor \cdot \rfloor$ means rounding down.

3. **Example:** with $p = 100$ features: “sqrt” gives $\sqrt{100} = 10$; “log2” gives $\log_2 100 = 6.64$, so **6**; 0.2 gives 20; `None` gives all 100. The Notebook confirms each one.

With the 2 features of our demo data, “sqrt” gives $\lfloor 1.41 \rfloor = 1$: each split may look at only one randomly chosen feature.

Extra: Older code and documentation also list `max_features="auto"`, which meant “sqrt” for the classifier and all the features for the regressor. The “auto” option was removed in scikit-learn 1.3 and now raises an error; the classifier’s default became “sqrt” in 1.1 and the regressor’s 1.0 (all features). (scikit-learn API docs)

3.3 Trying the settings on a demo dataset

Key point: More trees smooth the boundary until the score levels off; very few observations per tree hurt; the other settings change little on 2-feature data.

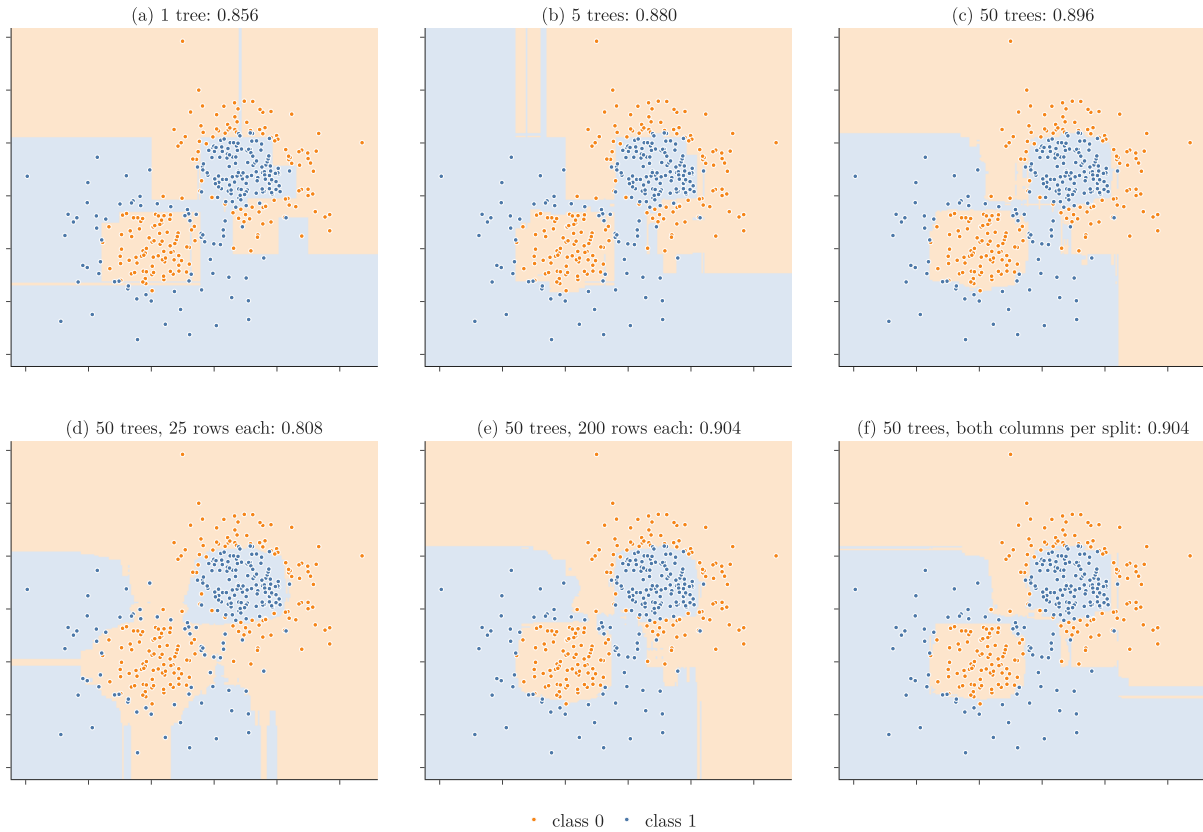


Figure 2: Random forests on the demo data with different forest-level settings; titles give the test accuracy

The demo data has 500 points with 2 features and 2 classes: two rings of points, each around a blob of the other class. We train on 375 points and test on 125. Figure 2 shows six forests on one such split; the Dash app lets us try any combination.

With only 125 test points, one point moves the accuracy by 0.008, so a single split is noisy. The numbers below are averages over 20 random splits (100 trees unless the setting says otherwise).

n_estimators. One tree scores 0.846, 5 trees 0.868 and 10 trees 0.888. Then the score stops rising: 50, 100 and 200 trees all score about 0.887. Adding trees to a forest does not cause overfitting; the score just levels off (ESL §15.3.4).

- With **few trees** the boundary is erratic, with odd strips and corners that show overfitting (Figure 2a, b).
- With **more trees** these areas smooth out (Figure 2c).
- Each extra tree costs training time, so beyond the point where the score stops rising, more trees only add cost.

max_samples. Very few observations per tree hurt: 25 observations score 0.827 (Figure 2d)

and 50 score 0.867. From 100 observations (about a quarter of 375) on, the score is flat at about 0.89 (100: 0.889, 200: 0.892, all 375: 0.887; Figure 2e). A tree grown on 25 points has too little data to find the rings, and averaging many such weak trees cannot fix that.

max_features. One feature per split (the default here) scores 0.887, both features 0.884 (Figure 2f). Even with one feature the forest does well, because each split gets a randomly chosen feature, so both features are used across the tree.

bootstrap. True scores 0.887, False 0.880. Without bootstrap every tree sees the same observations, so the trees are more alike and the average removes less variance (the [bagging vs random forest Note](#), section 3.3).

Extra: With `bootstrap=False`, every tree is trained on the whole training set: the observations are not drawn without replacement, they are not sampled at all. So `max_samples` cannot be set (scikit-learn raises a `ValueError`). If, on top of that, `max_features=None`, the only randomness left is tie-breaking between equally good splits (scikit-learn API docs, `DecisionTreeClassifier`, `random_state`), and the trees come out nearly identical (45 or 46 leaves each in the Notebook): a forest of copies of one tree.

4. The tree-level hyperparameters

Key point: Every tree in the forest takes the usual decision tree settings; they work exactly as for a single tree.

These settings are applied to each tree in the forest. Each is explained, with its effect on overfitting, in the [decision tree hyperparameters Note](#):

Setting	Default	What it controls
<code>criterion</code>	"gini"	how to measure split quality: "gini", "entropy" or "log_loss"
<code>max_depth</code>	None (fully grown)	the deepest a tree may grow
<code>min_samples_split</code>	2	the fewest observations a node needs before it may split
<code>min_samples_leaf</code>	1	the fewest observations allowed in a leaf
<code>min_weight_fraction_leaf</code>	0.0	the same as <code>min_samples_leaf</code> , as a share of all the observations
<code>max_leaf_nodes</code>	None	the most leaves a tree may have
<code>min_impurity_decrease</code>	0.0	the smallest impurity decrease worth a split

<code>ccp_alpha</code>	0.0 (no pruning)	cost-complexity pruning (Extra below)
------------------------	------------------	--

Most are pruning settings: they stop a tree before it fits every training point. In a random forest they are usually left at their defaults, because the trees are meant to be fully grown (low bias); the forest removes the variance (the [random forest and bias-variance Note](#)).

Extra: `ccp_alpha` prunes a grown tree back. Every subtree is scored by its training error plus `ccp_alpha` times its number of leaves, and the subtree with the lowest score is kept. A larger `ccp_alpha` charges more for each leaf, so the tree gets smaller.

In the Notebook, with 100 trees on the demo data (accuracy averaged over 20 splits): `ccp_alpha=0` gives 41.9 leaves per tree (0.887); 0.002 gives 39.2 (0.886); 0.01 gives 12.1 (0.878); 0.05 only 3.9, and the forest underfits (0.776). Pruning never helps here, which is why the trees of a forest are left fully grown.

5. The general hyperparameters

Key point: `oob_score` gives a free accuracy estimate, `n_jobs` speeds training up, `random_state` makes runs repeatable, `warm_start` adds trees, `class_weight` helps imbalanced data.

Setting	Default	Meaning
<code>oob_score</code>	<code>False</code>	score the forest on its out-of-bag observations (the OOB score Note)
<code>n_jobs</code>	<code>None</code> (1 core)	train trees in parallel on several CPU cores; -1 means all
<code>random_state</code>	<code>None</code>	fixes the random draws of observations and features, so the same settings give the same forest
<code>verbose</code>	0	print progress during training and prediction
<code>warm_start</code>	<code>False</code>	<code>True</code> keeps the trees already trained and adds new ones on the next <code>fit</code>
<code>class_weight</code>	<code>None</code>	weights for each class, for imbalanced data (as in the logistic regression hyperparameters Note)

Setting	Default	Meaning
<code>monotonic_cst</code>	None	forces predictions to only rise (1) or only fall (-1) as a feature grows

Two of them deserve a closer look:

- **random_state.** Without it, training the same forest twice gives slightly different results, because the observations and the features at every split are drawn at random. Fixing it, for example `random_state=42`, makes the results repeatable.
- **warm_start.** The forest can be trained in stages: train 50 trees, set `n_estimators=100`, call `fit` again, and only the 50 new trees are trained. Training in stages is handy for finding how many trees are enough without retraining from scratch.

Python: Adding trees with `warm_start`.

```
rf = RandomForestClassifier(n_estimators=50,
                           warm_start=True, random_state=42)
rf.fit(X_train, y_train)           # 50 trees
rf.set_params(n_estimators=100)
rf.fit(X_train, y_train)           # 50 more: 100 in total
```

`set_params` changes a setting of a model after it is created. The new trees are trained on the data passed to the second `fit`.

Extra: `monotonic_cst` (added in scikit-learn 1.4) takes one value per feature: 1, -1 or 0 (no constraint). For example, a model of house prices can be forced to never predict a lower price for a bigger house. The constraint works for regression and for two-class classification, not for multi-class or multi-output problems (scikit-learn API docs).

6. The random forest regressor

Key point: Same hyperparameters, except the criterion options and the `max_features` default.

`RandomForestRegressor` has the same settings as the classifier, with two differences:

- **criterion:** "squared_error" (the default; mean squared error, the variance reduction of the [regression trees Note](#)), "absolute_error" or "poisson", instead of Gini and entropy.
- **max_features:** the default is 1.0, all the features at every split, instead of "sqrt".

The regressor has no `class_weight`, since there are no classes.

Extra: Older code uses `criterion="mse"` and "mae"; these were renamed "squared_error" and "absolute_error" and the old names were removed in scikit-learn 1.2. A fourth option, "friedman_mse", is deprecated since 1.9. The setting

`min_impurity_split`, listed as deprecated in old documentation, was removed in 1.0; `min_impurity_decrease` replaces it (scikit-learn API docs).

7. Summary

Group	Settings	Usually
Forest	<code>n_estimators</code>	the default 100, or more until the score stops rising
	<code>max_samples</code>	the default (all observations); very small samples hurt
	<code>max_features</code>	“sqrt” (classifier) or all features (regressor); tune it
Each tree	<code>bootstrap</code>	True
	<code>criterion</code> , <code>max_depth</code> , <code>min_samples_split</code> , ...	defaults: fully grown trees
General	<code>n_jobs=-1</code> , <code>random_state</code> , <code>oob_score</code> , <code>warm_start</code> , <code>class_weight</code>	as needed

- On the demo data, accuracy rose from 0.846 (1 tree) to 0.888 (10 trees), then stayed flat.
- 25 observations per tree scored only 0.827; from 100 observations on, about 0.89.
- `bootstrap=False` means every tree gets all the observations; `max_samples` then cannot be set.
- The regressor differs only in its criteria and its `max_features` default.

8. Sources

- Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning*, 2nd ed. Springer. §15.3.4 (random forests and overfitting).
- scikit-learn developers. API reference pages for `RandomForestClassifier`, `RandomForestRegressor` and `DecisionTreeClassifier`, versions 0.24, 1.1 and 1.9 (version notes under each parameter).

9. Key terms

Term	Meaning
Forest-level hyperparameters	The settings that shape the forest itself: <code>n_estimators</code> , <code>max_features</code> , <code>bootstrap</code> , <code>max_samples</code>
<code>ccp_alpha</code>	Cost-complexity pruning strength: the penalty per leaf when a grown tree is pruned back
<code>warm_start</code>	Setting that keeps already-trained trees and adds new ones on the next fit

Term	Meaning
<code>set_params</code>	Method that changes a model's settings after it is created
<code>monotonic_cst</code>	Setting that forces predictions to only rise or only fall as a feature grows